

Java Servlet 數學運算計算器 - 完整教學文件

📖 教學目標

本教學專案將帶領您學習 Java Servlet 的核心概念，透過實作一個數學運算計算器來掌握：

- Servlet 的基本架構和生命週期
- HTTP 請求和回應的處理
- 表單數據的接收和驗證
- 錯誤處理和例外管理
- Web 應用程式的部署配置

🏠 專案結構

```
servlet-math-calculator/
├── pom.xml                    # Maven 專案配置文件
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/
│   │   │   │   ├── example/
│   │   │   │   │   ├── servlet/
│   │   │   │   │   │   ├── MathOPServlet.java        # 主要 Servlet 類別
│   │   │   │   │   │   └── CharacterEncodingFilter.java # 編碼過濾器
│   │   └── webapp/
│   │       ├── index.html        # 前端表單頁面
│   │       ├── error/
│   │       │   ├── 404.html
│   │       │   └── 500.html
│   │       └── WEB-INF/
│   │           └── web.xml        # Web 應用程式配置
└── README.md                    # 本教學文件
```

🔗 核心功能

1. 數學運算支援

- **加法 (Addition)**：兩數相加
- **減法 (Subtraction)**：第一個數減去第二個數
- **乘法 (Multiplication)**：兩數相乘
- **除法 (Division)**：第一個數除以第二個數 (含除零檢查)

2. 輸入驗證

- 第一個數字：範圍 10-20
- 第二個數字：範圍 1-10
- 前端 JavaScript 驗證 + 後端 Java 驗證

3. 錯誤處理

- 數字格式錯誤處理
- 除零錯誤處理
- 參數缺失處理
- 美觀的錯誤頁面顯示

💡 Servlet 核心概念解析

1. Servlet 生命週期

```
// Servlet 的三個主要階段
public class MathOPServlet extends HttpServlet {

    // 1. 初始化階段 ( 只執行一次 )
    @Override
    public void init() throws ServletException {
        // Servlet 容器載入時執行
        // 可在此進行初始化設定
    }

    // 2. 服務階段 ( 每次請求都執行 )
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse
response) {
        // 處理客戶端請求
        // 本專案的核心邏輯都在這裡
    }

    // 3. 銷毀階段 ( 容器關閉時執行 )
    @Override
    public void destroy() {
        // 清理資源
    }
}
```

2. HTTP 請求處理

```
// 接收表單參數
String no1Str = request.getParameter("no1");           // 第一個數字
String no2Str = request.getParameter("no2");           // 第二個數字
String operation = request.getParameter("submit");      // 運算類型

// 設定編碼處理中文
request.setCharacterEncoding("UTF-8");
response.setContentType("text/html;charset=UTF-8");
```

3. 回應生成

```
// 取得輸出流
PrintWriter out = response.getWriter();

// 輸出 HTML 內容
out.println("<!DOCTYPE html>");
out.println("<html lang='zh-TW'>");
out.println("<head><meta charset='UTF-8'><title>結果</title></head>");
out.println("<body>");
out.println("<h1>運算結果 : " + result + "</h1>");
out.println("</body>");
out.println("</html>");

// 記得關閉流
out.close();
```

關鍵程式碼解析

1. MathOPServlet.java - 主要邏輯

```
@WebServlet("/MathOPServlet") // 註解式 URL 映射
public class MathOPServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {

        try {
            // 步驟 1: 接收和驗證參數
            String no1Str = request.getParameter("no1");
            String no2Str = request.getParameter("no2");
            String operation = request.getParameter("submit");

            // 步驟 2: 轉換數據類型
            double num1 = Double.parseDouble(no1Str.trim());
            double num2 = Double.parseDouble(no2Str.trim());

            // 步驟 3: 執行運算
            double result = performCalculation(num1, num2, operation);

            // 步驟 4: 顯示結果
            showResult(out, num1, num2, operation, result);

        } catch (NumberFormatException e) {
            showError(out, "數字格式錯誤");
        } catch (ArithmeticException e) {
            showError(out, "數學運算錯誤");
        }
    }
}
```

2. 運算邏輯實作

```
private double performCalculation(double num1, double num2, String operation) {
    switch (operation.toLowerCase()) {
        case "add":
            return num1 + num2;           // 加法

        case "subtract":
            return num1 - num2;           // 注意原表單的拼字
            // 減法

        case "multiply":
            return num1 * num2;           // 乘法

        case "divide":
            if (num2 == 0) {
                throw new ArithmeticException("除數不能為零!");
            }
            return num1 / num2;           // 除法

        default:
            throw new IllegalArgumentException("不支援的運算");
    }
}
```

3. 前端表單設計

```
<form action="MathOPServlet" method="post">
  <!-- 數字輸入欄位 -->
  <input type="number" name="no1" min="10" max="20" value="15" required>
  <input type="number" name="no2" min="1" max="10" value="5" required>

  <!-- 運算按鈕 -->
  <input type="submit" name="submit" value="add">
  <input type="submit" name="submit" value="subtract">
  <input type="submit" name="submit" value="multiply">
  <input type="submit" name="submit" value="divide">
</form>
```



部署和執行步驟

方法一：使用 Maven + Tomcat

1. 編譯專案

```
cd servlet-math-calculator
mvn clean compile
```

2. 打包成 WAR 檔案

```
mvn package
```

3. 部署到 Tomcat

```
# 將 target/servlet-math-calculator-1.0.0.war 複製到 Tomcat 的 webapps 目錄  
cp target/servlet-math-calculator-1.0.0.war $TOMCAT_HOME/webapps/
```

4. 啟動 Tomcat

```
$TOMCAT_HOME/bin/startup.sh    # Linux/Mac  
$TOMCAT_HOME/bin/startup.bat   # Windows
```

5. 訪問應用程式

```
http://localhost:8080/servlet-math-calculator-1.0.0/
```

方法二：使用 IDE (Eclipse/IntelliJ)

1. 匯入 Maven 專案

- File → Import → Existing Maven Projects
- 選擇專案根目錄

2. 配置伺服器

- 右鍵專案 → Run As → Run on Server
- 選擇 Tomcat 伺服器

3. 執行測試

- 瀏覽器會自動開啟應用程式



測試案例

1. 正常運算測試

輸入1	輸入2	運算	預期結果
15	5	add	20
15	5	subtract	10

輸入1	輸入2	運算	預期結果
15	5	multiply	75
15	5	divide	3.00

2. 邊界值測試

輸入1	輸入2	說明
10	1	最小值
20	10	最大值
15	0	除零測試

3. 錯誤處理測試

- 空白輸入
- 非數字輸入
- 超出範圍的數字
- 除以零的情況

學習重點

1. Servlet 基礎概念

- **什麼是 Servlet ?** Servlet 是運行在 Web 伺服器上的 Java 程式，用於處理客戶端請求並生成動態回應。
- **為什麼使用 Servlet ?**
 - 平台無關性
 - 效能優異 (共享 JVM 實例)
 - 強大的 API 支援
 - 與 Java EE 生態系統整合

2. HTTP 通訊協定

- **GET vs POST**
 - GET：用於取得資料，參數在 URL 中
 - POST：用於提交資料，參數在請求主體中
- **請求/回應循環** 客戶端 → HTTP 請求 → Servlet 處理 → HTTP 回應 → 客戶端

3. 最佳實務

- ☒ 總是驗證輸入參數
- ☒ 適當的例外處理
- ☒ 設定正確的編碼
- ☒ 關閉資源 (Stream、Connection)
- ☒ 使用有意義的錯誤訊息

進階擴展

1. 新增更多運算功能

```
case "power":
    return Math.pow(num1, num2);    // 次方運算

case "mod":
    return num1 % num2;             // 餘數運算

case "sqrt":
    return Math.sqrt(num1);         // 平方根
```

2. 新增運算歷史記錄

```
// 使用 Session 儲存歷史
HttpSession session = request.getSession();
List<String> history = (List<String>) session.getAttribute("history");
if (history == null) {
    history = new ArrayList<>();
    session.setAttribute("history", history);
}
history.add(num1 + " " + operation + " " + num2 + " = " + result);
```

3. 資料庫整合

```
// 使用 JDBC 儲存運算記錄
String sql = "INSERT INTO calculations (num1, num2, operation, result, timestamp)
VALUES (?, ?, ?, ?, ?)";
try (PreparedStatement pstmt = connection.prepareStatement(sql)) {
    pstmt.setDouble(1, num1);
    pstmt.setDouble(2, num2);
    pstmt.setString(3, operation);
    pstmt.setDouble(4, result);
    pstmt.setTimestamp(5, new Timestamp(System.currentTimeMillis()));
    pstmt.executeUpdate();
}
```

延伸學習

1. 相關技術

- **JSP (JavaServer Pages)**：更適合視圖層的技术
- **Spring MVC**：現代化的 Web 框架
- **RESTful API**：API 設計最佳實務
- **JSON 處理**：現代 Web 應用的數據格式

2. 架構模式

- **MVC 模式**：Model-View-Controller
- **Front Controller 模式**：統一請求入口
- **DAO 模式**：數據存取物件

3. 安全性考量

- **SQL Injection 防護**：使用 PreparedStatement
- **XSS 防護**：輸出編碼
- **CSRF 防護**：Token 驗證

常見問題 FAQ

Q1: 為什麼使用 POST 而不是 GET ?

A: POST 更適合提交表單數據，且不會在 URL 中暴露敏感資訊。GET 應該用於取得資料且具有冪等性。

Q2: @WebServlet 和 web.xml 配置有什麼差別？

A:

- **@WebServlet**：註解式配置，簡潔但編譯時決定
- **web.xml**：XML 配置，靈活且可動態修改

Q3: 如何處理中文編碼問題？

A:

```
// 設定請求編碼
request.setCharacterEncoding("UTF-8");
// 設定回應編碼
response.setContentType("text/html; charset=UTF-8");
```

Q4: Servlet 是線程安全的嗎？

A: Servlet 實例是單例的，多個請求會共享同一實例，因此需要注意線程安全。避免使用實例變數儲存請求相關的數據。

學習評估

完成本教學後，您應該能夠：

- ☐ 解釋 Servlet 的生命週期
- ☐ 處理 HTTP 請求和回應
- ☐ 實作表單數據處理
- ☐ 設計適當的錯誤處理機制
- ☐ 配置 Web 應用程式部署描述符
- ☐ 理解 HTTP 通訊協定基礎

- ☐ 應用 MVC 設計模式概念

技術支援

如果您在學習過程中遇到問題，可以：

1. **查看日誌檔案**：檢查 Tomcat logs 目錄下的錯誤訊息
2. **使用除錯工具**：IDE 的除錯功能或瀏覽器開發者工具
3. **參考官方文檔**：Oracle Java EE 文檔和 Apache Tomcat 文檔
4. **社群資源**：Stack Overflow、GitHub、技術論壇

作者：程式設計教學教授

版本：1.0

最後更新：2025年10月

授權：教育用途免費使用

專案原始碼：完整的程式碼已包含在本專案中，歡迎研究和改進！